



UNIVERSITATEA DIN CRAIOVA

Facultatea de Automatica, Calculatoare si Electronica

Specializarea: Automatica si Informatica Aplicata

Grupa: 2.1 A

PROIECT la disciplina BAZE DE DATE

“..... Salon de coafura”



Echipa: RB ORACLE

Membri echipei:

CIMPUREANU ANDA - MARIA

CALCIU IONELA - ALESSANDRA

Coordonator, Prof. Dr. Ing. Viorel Stoian

Craiova, 2026

CUPRINS

CAPITOLUL I. INTRODUCERE.....	3
CAPITOLUL II. TEMA DE PROIECT.....	4
CAPITOLUL III. SCHEMA CONCEPTUALA.....	5
3.1. Notiuni teoretice.....	5
3.2. Schema conceptuala.....	6
CAPITOLUL IV. SCHEMA LOGICA.....	6
4.1. Notiuni teoretice.....	6
4.2. Schema logica.....	8
CAPITOLUL V. NORMALIZAREA BD.....	11
5.1. Notiuni teoretice.....	11
5.2. Normalizarea tabelelor bazei de date.....	13
CAPITOLUL VI. DENORMALIZAREA BD.....	14
6.1. Notiuni teoretice.....	14
6.2. Denormalizarea bazei de date.....	15
CAPITOLUL VII. SGBD utilizat.....	15
7.1. Notiuni teoretice.....	15
7.2. Exemple de interogari SQL pe baza de date create.....	17
CAPITOLUL VIII. CONCLUZII.....	18
BIBLIOGRAFIE.....	19

CAPITOLUL I. INTRODUCERE

1.1. Generalități și importanța bazelor de date

Bazele de date și sistemele de gestiune a acestora reprezintă o componentă esențială a societății moderne. Majoritatea activităților zilnice implică interacțiunea cu o bază de date, fie că este vorba despre operațiuni bancare, rezervări la hoteluri, accesarea unui catalog de bibliotecă sau cumpărături online. Chiar și simpla achiziție a produselor într-un supermarket actualizează automat baza de date a inventarului.

Pe lângă aplicațiile tradiționale (unde informația este text sau numerică), avansul tehnologic a dus la apariția unor noi tipuri de baze de date. Rețelele de socializare (Facebook, Twitter) gestionează volume uriașe de date netradiționale (imagini, video, mesaje), necesitând sisteme de tip „Big Data” sau NoSQL. De asemenea, sistemele de informații geografice (GIS) sau cele multimedia stochează și analizează date complexe precum hărți, fluxuri video sau imagini satelitare.

O bază de date are următoarele proprietăți implicite:

1. O bază de date trebuie să reprezinte anumite aspecte ale lumii reale, numită univers de discurs - universe of discourse (UoD), sau microunivers - miniworld
2. O bază de date trebuie să fie o colecție logic-coerentă de date, cu un înțeles inherent. O colecție aleatoare de date nu poate fi corect numită o bază de date
3. O bază de date trebuie să fie proiectată, construită și populată cu date pentru un scop specific. Are un grup țintă de utilizatori și unele aplicații preconcepute în care acești utilizatori sunt interesați.

O bază de date are:

- o sursă de unde sunt derivate datele,
- un anumit grad de interacțiune cu evenimentele din lumea reală
- un public activ interesat de conținutul său.

Utilizatorii finali ai unei baze de date pot efectua diverse acțiuni, care determină schimbarea informațiilor din baza de date. Pentru ca o bază de date să fie

precisă și de încredere la orice moment de timp, aceasta trebuie să reflecte corect microuniversul pe care îl reprezintă.

O bază de date poate fi creată (definită) și gestionată fie manual, fie computerizat.

Un sistem de gestionare a bazelor de date (DBMS - DataBase Management System) este un sistem computerizat care permite utilizatorilor să creeze și să mențină o bază de date.

Definirea unei baze de date implică specificarea tipurilor de date, structurilor și constrângerilor datelor care vor fi stocate în bază de date.

Construirea bazei de date este procesul de stocare a datelor pe un suport de stocare controlat de DBMS.

Manipularea unei baze de date include funcții precum interogarea bazei de date pentru a recupera date specifice, actualizarea bazei de date pentru a reflecta schimbările din mini-lume și generarea de rapoarte din date.

Un program, sau o aplicație, accesează baza de date trimițând interogări sau solicitări de date către DBMS.

O interogare, de obicei, determină recuperarea unor date;

O tranzacție poate determina citirea unor date și scrierea unor date în bază de date.

O bază de date poate fi de orice dimensiune și complexitate.

CAPITOLUL II. TEMA DE PROIECT

Saloane de coafură

În salonul nostru, avem o serie de coafori. Toți sunt angajați salariați, așa că ținem o evidență a numelui, prenumelui, adresei, numărului de telefon, numărului de securitate socială (cnp) și salariului. Pe parcursul unei zile, un coafor poate avea mai mulți clienți. Într-o zi puțin aglomerată, este posibil ca un coafor să nu aiba niciun client. Avem mai mulți clienți walk-in (care intra - neprogramati) și fiecare este repartizat la un coafor. Le cerem doar prenumele. Avem și clienți care sună pentru a face o programare. Când fac acest lucru, le cerem numele, prenumele și numărul de telefon. Întrebăm, de asemenea, dacă le-ar plăcea un stilist specific. Dacă nu au nicio preferință, le atribuim noi unul. Desigur, li se permite să treacă la un alt coafor pentru

următoarea lor vizită la salon. Suntem interesați să urmărim întâlnirile zilnice - care stilist lucrează la ce client în timpul unei zile date.

Salonul prezentat mai sus este doar unul dintr-un lant de saloane ale aceleiasi companii, distribuite in mai multe orase ale Romaniei. Baza de date trebuie sa stocheze informatii despre locatiile saloanelor si datele de contact. Trebuie sa se tina o evidenta a operatiilor (serviciilor) respective asupra clientilor. Fiecare salon este dotat cu o serie de instrumente și materiale, substanțe specifice care sunt utilizate de coafori. Baza de date trebuie să stocheze informații despre aceste instrumente și materiale, cum ar fi: denumirea, operația realizată, perioada de garanție, data când se va casa. De asemenea, baza de date trebuie să stocheze informații despre furnizori, dar și despre fabricile producătoare. Trebuie să se țină o evidență a achizițiilor (cumpărăturilor) de instrumente notându-se: data, numărul facturii, produsul cumpărat, prețul, cantitatea, furnizorul, producătorul, angajatul care s-a ocupat de achiziția respectivă.

CAPITOLUL III. SCHEMA CONCEPTUALA

3.1. Notiuni teoretice

Modelul Entitate-Legătură Proiectarea unei baze de date începe cu realizarea schemei conceptuale, utilizându-se modelul entitate-legătură. Principalele concepte folosite în acest model sunt: entitatea, relația (legătura) și atributul.

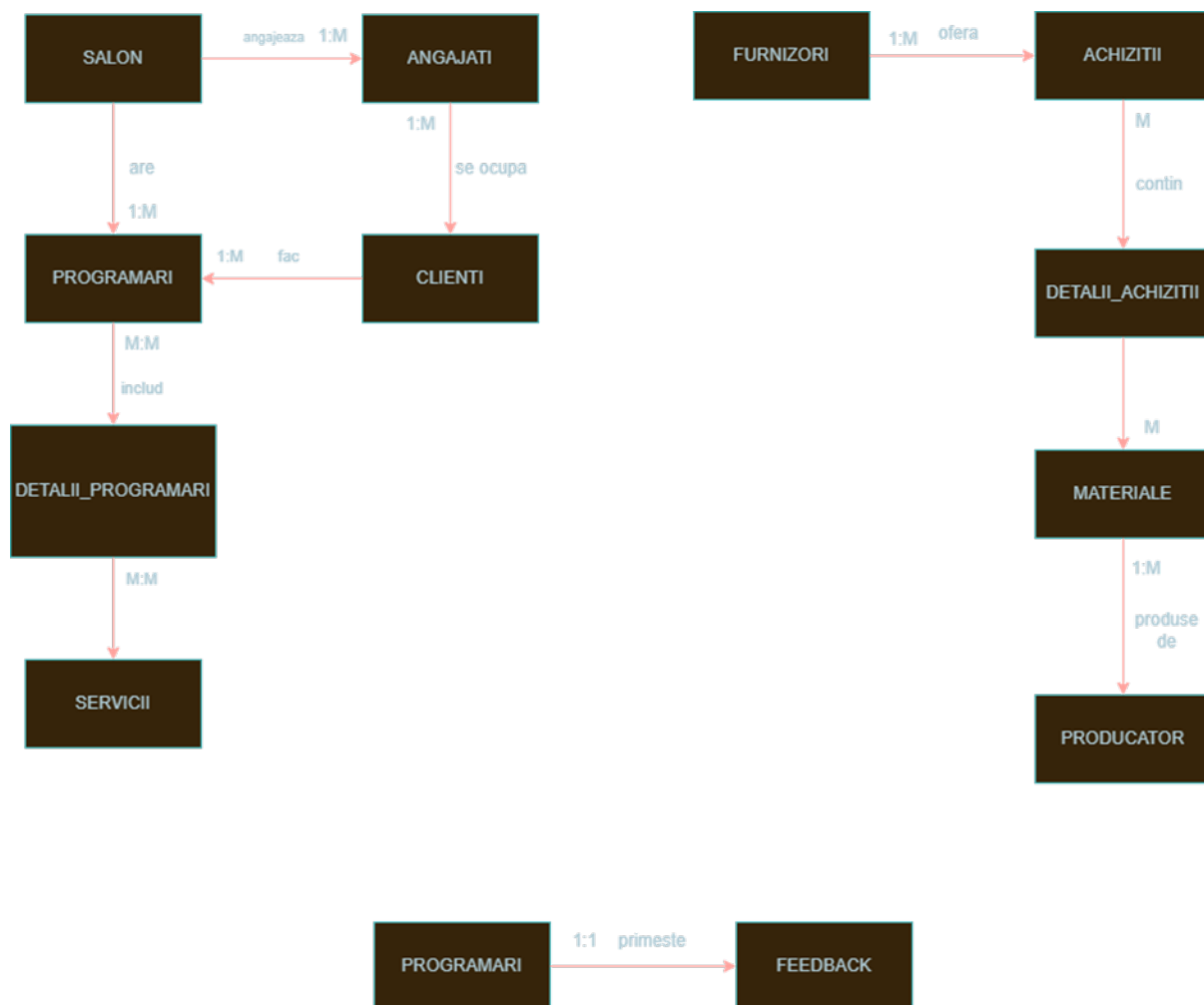
Entitatea : Este un obiect de interes din sistem pentru care trebuie să existe date înregistrate. Entitățile sunt reprezentate grafic prin dreptunghiuri și au o denumire unică în cadrul sistemului.

Relația (Legătura): Este o asociere nedirecționată între două entități. Cardinalitatea unei relații indică numărul de instanțe din fiecare entitate care pot participa la relație. Există trei tipuri de cardinalitate: "unu-la-unu" (1:1), "mulți-la-unu" (M:1) și "mulți-la-mulți" (M:M).

Atributul : Este o caracteristică a unei entități sau a unei relații. Fiecare atribut are un nume unic și furnizează informații despre entitatea respectivă (ex: nume, dată).

Cheia Primară: Este un atribut sau un set de attribute care identifică în mod unic o instanță a entității, făcând distincție între oricare două rânduri diferite

3.2. Schema conceptuala



CAPITOLUL IV. SCHEMA LOGICA

4.1. Notiuni teoretice

Realizarea schemei logice a unei baze de date reprezintă etapa de tranziție de la modelul conceptual (diagrama entitate-legătură) la implementarea efectivă în modelul relațional. Această etapă presupune conversia entităților și a legăturilor identificate anterior în tabele (relații) bidimensionale.

Procesul de transformare se bazează pe un set de reguli standardizate de mapare:

1. Transformarea Entităților Regula generală stipulează că fiecare entitate din diagrama conceptuală se transformă într-un tabel în schema logică.

- **Entitățile independente** devin tabele independente, având o cheie primară proprie care nu conține chei străine.
- **Entitățile dependente** devin tabele care includ în structura lor o cheie străină, realizând astfel legătura cu entitatea părinte.

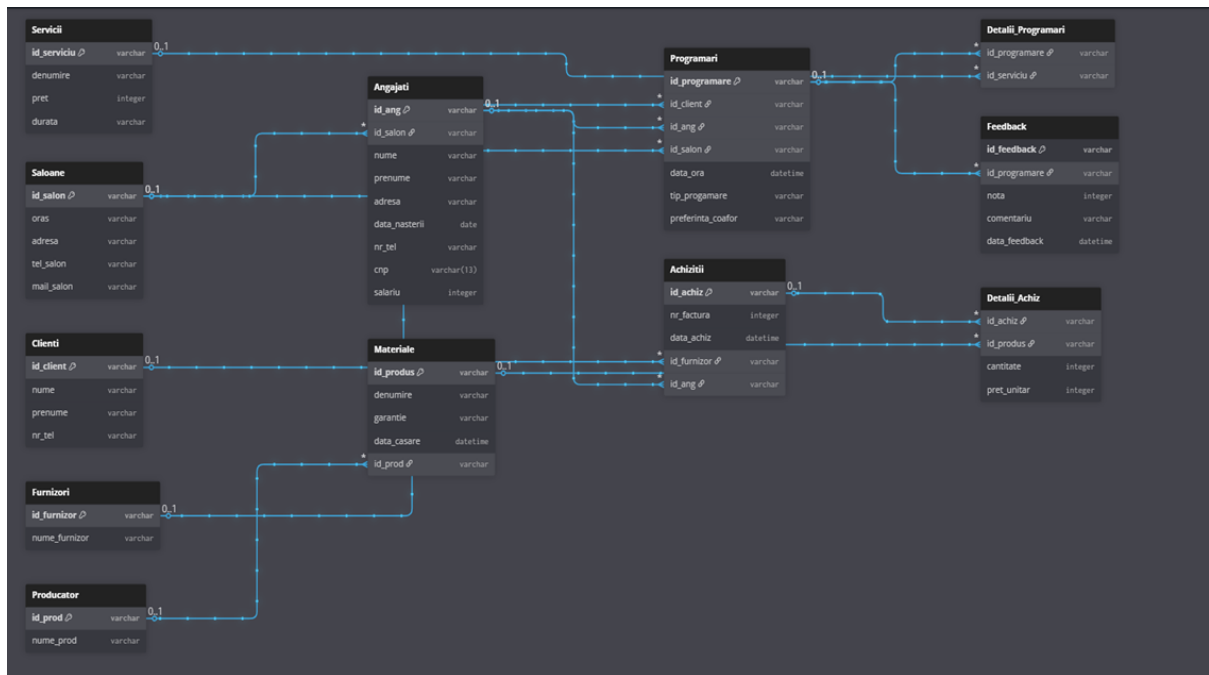
2. Transformarea Relațiilor (Legăturilor) Relațiile dintre entități se convertesc, de regulă, prin mecanismul cheilor străine (Foreign Keys). Regulile specifice în funcție de cardinalitate sunt:

- **Relațiile de tip „unu-la-mulți” (1:M):** Se transformă prin adăugarea unei chei străine în tabelul corespunzător entității aflate pe partea de „mulți” a relației. Această cheie străină referențiază cheia primară a entității de pe partea „unu”.
- **Relațiile de tip „mulți-la-mulți” (M:M):** Acestea nu pot fi implementate direct prin chei străine simple. Ele se transformă în **tabele asociative** (de legătură). Un tabel asociativ va conține cel puțin două chei străine, care fac referință la cheile primare ale celor două tabele inițiale implicate în relație. Cheia primară a acestui nou tabel este formată, de obicei, din combinația celor două chei străine.
- **Relațiile de tip „unu-la-unu” (1:1):** Se transformă prin plasarea cheii străine (care trebuie să fie unică) în unul dintre cele două tabele, de obicei în cel care are mai puține înregistrări sau care este considerat secundar.

3. Transformarea Atributelor Fiecare atribut al unei entități devine o coloană în tabelul rezultat.

- Cheile primare ale entităților devin cheile primare ale tabelelor.
- Atributele compuse sunt descompuse în coloane distincte pentru fiecare componentă.

4.2. Schema logica



Lista tabelelor rezultate:

```

CREATE TABLE Saloane (
    id_salon VARCHAR(100) PRIMARY KEY NOT NULL,
    oras VARCHAR(100) NOT NULL,
    adresa VARCHAR(255) NOT NULL,
    tel_salon VARCHAR(20) NOT NULL,
    mail_salon VARCHAR(100) NOT NULL
);
  
```

```

CREATE TABLE Clienti (
    id_client VARCHAR(100) PRIMARY KEY NOT NULL,
    nume VARCHAR(50) NOT NULL,
    prenume VARCHAR(50) NOT NULL,
    nr_tel VARCHAR(20) NOT NULL
);
  
```

```

CREATE TABLE Servicii (
    id_serviciu VARCHAR(100) PRIMARY KEY NOT NULL,
  
```



```
denumire VARCHAR(100) NOT NULL,  
pret DECIMAL(10, 2) NOT NULL,  
durata VARCHAR(50) NOT NULL  
);
```

```
CREATE TABLE Furnizori (  
    id_furnizor VARCHAR(100) PRIMARY KEY NOT NULL,  
    nume_furnizor VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Producator (  
    id_prod VARCHAR(100) PRIMARY KEY NOT NULL,  
    nume_prod VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Angajati (  
    id_ang VARCHAR(100) PRIMARY KEY NOT NULL,  
    id_salon VARCHAR(100) NOT NULL,  
    nume VARCHAR(50) NOT NULL,  
    prenume VARCHAR(50) NOT NULL,  
    adresa VARCHAR(255) NOT NULL,  
    data_nasterii DATE NOT NULL,  
    nr_tel VARCHAR(20) NOT NULL,  
    cnp VARCHAR(13) NOT NULL,  
    salariu INT NOT NULL,  
);
```

```
CREATE TABLE Materiale (  
    id_produs VARCHAR(100) PRIMARY KEY NOT NULL,  
    denumire VARCHAR(100) NOT NULL,  
    garantie VARCHAR(50) NOT NULL,  
    data_casare DATETIME NOT NULL,
```

```
id_prod VARCHAR(100) NOT NULL,  
);
```

```
CREATE TABLE Programari (  
    id_programare VARCHAR(100) PRIMARY KEY NOT NULL,  
    id_client VARCHAR(100) NOT NULL,  
    id_ang VARCHAR(100) NOT NULL,  
    id_salon VARCHAR(100) NOT NULL,  
    data_ora DATETIME NOT NULL,  
    tip_programare VARCHAR(50) NOT NULL,  
    preferinta_coafor VARCHAR(10) NOT NULL,  
);
```

```
CREATE TABLE Detalii_Programari (  
    id_programare VARCHAR(100) NOT NULL,  
    id_serviciu VARCHAR(100) NOT NULL,  
    PRIMARY KEY (id_programare, id_serviciu)  
);
```

```
CREATE TABLE Achizitii (  
    id_achiz VARCHAR(100) PRIMARY KEY NOT NULL,  
    nr_factura INT NOT NULL,  
    data_achiz DATETIME NOT NULL,  
    id_furnizor VARCHAR(100) NOT NULL,  
    id_ang VARCHAR(100) NOT NULL  
);
```

```
CREATE TABLE Detalii_Achiz (  
    id_achiz VARCHAR(100) NOT NULL,  
    id_produs VARCHAR(100) NOT NULL,  
    cantitate INT NOT NULL,  
    pret_unitar DECIMAL(10, 2) NOT NULL,
```

```

PRIMARY KEY (id_achiz, id_produș),
);
CREATE TABLE Feedback (
    id_feedback INT PRIMARY KEY,
    id_programare VARCHAR(100) NOT NULL,
    nota INT CHECK (nota BETWEEN 1 AND 5),
    comentariu VARCHAR(255),
    data_feedback DATETIME NOT NULL,
    FOREIGN KEY (id_programare) REFERENCES Programari(id_programare)
);

```

CAPITOLUL V. NORMALIZAREA BD

5.1. Notiuni teoretice

În trecut normalizarea era utilizată pentru proiectarea unei BD. În prezent, proiectarea unei BD se realizează pe baza celor prezentate anterior (schema conceptuală, schema logică),

iar normalizarea intervine asupra tabelelor obținute pe baza schemei logice eliminând unele

probleme care pot apărea în procesul de proiectare inițial: redundanța în date, anomalii la

actualizare.

Definiție: Normalizarea reprezintă procesul de descompunere a unui tabel relațional în

mai multe tabele care satisfac anumite reguli și care stochează aceleași date ca și tabelul inițial

astfel încât să fie eliminate redundanța în date și anomaliile la actualizare.

a) Caracterul reversibil al normalizării.

Prin caracter reversibil al normalizării se înțelege faptul că descompunerea se face fără

pierdere de informație, adică tabelul inițial poate fi reconstituit prin compunerea naturală, pe

atribute comune, a tabelelor rezultate.

b) Dependenta functionala

Definitie: Fie R un tabel relational si X si Y doua submultimi de coloane ale lui R.

Spunem ca X determina functional pe Y sau ca Y depinde functional de X daca nu exista doua

randuri in tabelul R care sa aiba aceleasi valori pentru coloanele din X si sa aiba valori diferite pentru cel putin o coloana din Y.

Obs.: Existenta dependentelor functionale pentru care determinantul nu este cheie a tabelului duce la aparitia redundantei in date si a anomalilor la actualizare in lucrul cu BD.

Definitie: O dependenta functionala care nu este totala se numeste dependenta functionala partiala.

Un tabel relational se poate afla in 6 situatii diferite numite forme normale:

- prima forma normala,
- a 2-a forma normala,
- a 3-a forma normala,
- forma normala Boyce-Codd,
- a 4-a forma normala,
- a 5-a forma normala.

Prima forma normala (1NF - First Normal Form)

Definitie: Un tabel relational este in 1NF daca fiecarei coloane ii corespunde o valoare indivizibila (atomica). Deci orice valoare nu poate fi compusa dintr-o multime de valori (atributele compuse) si nu sunt admise attributele repetitive.

A doua forma normala (2NF - Second Normal Form)

Definitie: Un tabel relational R este in a doua forma normala (2NF) daca si numai daca:

- R este in 1NF
- Orice coloana care depinde partial de o cheie a lui R este inclusa in acea cheie.

Deci, 2NF nu permite dependentele functionale partiale fata de cheile tabelului, cu exceptia dependentelor triviale, de incluziune.

A treia forma normala (3NF - Third Normal Form)

Definitie_1: Un tabel relational R este in a treia forma normala (3NF) daca si numai daca:

- R este in 2NF
- Pentru orice coloana A necontinuta in nici o cheie a lui R, daca exista un set de coloane X

Definitie_2: Un tabel relational R este in a treia forma normala (3NF) daca si numai daca:

- R este in 2NF
- Orice coloana necontinuta in nici o cheie a lui R nu este dependenta tranzitiv de nici o cheie a lui R.

5.2. Normalizarea tabelelor bazei de date

A. Analiza Formei Normale 1 (FN1) Un tabel respectă FN1 dacă toate atributele sale conțin valori atomice (indivizibile) și nu există grupuri repetitive.

- **Tabelele : SALOANE, CLIENȚI, SERVICII, FURNIZORI, PRODUCĂTOR, FEEDBACK** - respectă FN1, deoarece conțin doar date scalare (nume, adrese, date calendaristice) unice per înregistrare.
- **Tabelul ANGAJAȚI** - respectă FN1. Deși un angajat ar putea avea mai multe numere de telefon, în structura aleasă s-a optat pentru stocarea unui singur număr de contact principal, asigurând atomicitatea.
- **Tabelele : PROGRAMĂRI, ACHIZIȚII, MATERIALE** - respectă FN1, toate câmpurile fiind elementare.

B. Analiza Formei Normale 2 (FN2) Un tabel respectă FN2 dacă este în FN1 și toate atributele care nu fac parte din cheia primară depind total de întreaga cheie primară (nu există dependențe parțiale).

- **Tabelele cu cheie primară simplă (Saloane, Angajați, Clienți, Servicii, Programări, Materiale, Furnizori, Achiziții, Producători, Feedback):** Aceste tabele **respectă automat FN2**, deoarece cheia primară este formată dintr-o singură coloană (ex: id_salon, id_ang), deci dependențele parțiale sunt imposibile.
- **Tabelele cu cheie primară compusă (Tabelele Asociative):**
 - o **Tabelul DETALII_ACHIZ:** Are cheia primară compusă din (id_achiz, id_produs). Atributele non-cheie (cantitate, pret_unitar) depind de

ambele componente ale cheii (cantitatea se referă la un anumit produs de pe o anumită factură). Prin urmare, **tabelul respectă FN2**.

- o **Tabelul DETALII_PROGRAM_ĂRI:** Are cheia primară compusă din (id_programare, id_serviciu). Nu conține alte attribute non-cheie care să depindă parțial, deci **respectă FN2**.

C. Analiza Formei Normale 3 (FN3) Un tabel respectă FN3 dacă este în FN2 și nu există dependențe tranzitive (un atribut non-cheie nu depinde de alt atribut non-cheie).

- **Tabelul ANGAJAȚI:** Respectă FN3. Salariul și funcția depind direct de id_ang. Nu am inclus nume_salon sau adresa_salon în acest tabel (ceea ce ar fi creat o dependență tranzitivă), ci doar cheia externă id_salon.
- **Tabelul MATERIALE:** Respectă FN3. Datele despre producător (nume_prod) sunt stocate în tabelul separat **PRODUCĂTOR**, legătura făcându-se doar prin id_prod. Astfel, am eliminat redundanța și dependențele tranzitive.
- **Restul tabelelor (Saloane, Clienți, Servicii, etc.):** Respectă FN3, deoarece toate attributele descriptive depind direct și exclusiv de cheia primară a tabelului respectiv.

CAPITOLUL VI. DENORMALIZAREA BD

6.1. Notiuni teoretice

Denormalizarea unei baze de date reprezintă procesul invers operației de normalizare și duce la creșterea redundanței datelor. Prin aceasta se dorește, în principal, creșterea performanței și simplificarea programelor de interogare a datelor.

Obs:

- Denormalizarea se face numai după o normalizare corectă.
- Denormalizarea se face printr-o selectare strategică a structurilor care aduc avantaje semnificative
- Denormalizarea trebuie însoțită de măsuri suplimentare de asigurare a integrității datelor.

6.2. Denormalizarea bazei de date

În urma analizei cerințelor, baza noastră de date „Saloane de coafură”, nu necesită denormalizare.

CAPITOLUL VII. SGBD utilizat

7.1. Notiuni teoretice

Un sistem de baze de date reprezintă, în esență, o modalitate organizată de gestionare a unor volume mari de informații. Aceste informații pot proveni din surse variate, precum date de cercetare, înregistrări comerciale, cereri ale clienților, statistici sportive, rapoarte de vânzări, evidențe de personal sau rezultate școlare. Gestionarea eficientă a acestor date presupune nu doar stocarea lor, ci și posibilitatea de acces rapid, modificare controlată și protecție împotriva pierderilor sau accesului neautorizat.

MySQL este un sistem de gestiune a bazelor de date relaționale (SGBDR), care utilizează limbajul SQL (Structured Query Language) pentru definirea, manipularea și interogarea datelor. Acesta este unul dintre cele mai populare SGBD-uri datorită performanței ridicate, costului redus și compatibilității cu o gamă largă de tehnologii.

MySQL oferă o serie de avantaje semnificative:

Performanță ridicată: MySQL este cunoscut pentru viteza sa, în special în aplicațiile web și în sistemele cu un volum mare de interogări de citire.

Ușurință în utilizare: Instalarea, configurarea și administrarea sunt relativ simple comparativ cu alte sisteme enterprise, ceea ce îl face potrivit pentru utilizare educațională și proiecte de dimensiuni medii.

Cost redus: MySQL este disponibil gratuit pentru majoritatea utilizărilor, existând și ediții comerciale pentru organizații care necesită suport avansat.

Compatibilitate cu SQL: MySQL suportă limbajul SQL standard și poate fi accesat prin protocoale precum ODBC, permițând integrarea cu numeroase aplicații.

Suport pentru mai multe limbaje de programare: MySQL oferă conectori pentru limbaje precum C, Java, PHP, Python sau Perl, fiind ușor de integrat în aplicații software.

Securitate și control al accesului: Sistemul permite definirea de utilizatori, roluri și privilegii, precum și criptarea conexiunilor.

Portabilitate: MySQL rulează pe multiple sisteme de operare, inclusiv Linux și Windows, fiind utilizat frecvent pe servere Linux datorită stabilității și performanței acestora.

O bază de date este un ansamblu organizat de date stocate electronic, iar în MySQL aceasta este structurată sub forma unuiu sau mai multor tabele. Un tabel este o structură bidimensională formată din coloane (attribute) și rânduri (înregistrări). Fiecare coloană reprezintă un tip de informație, iar fiecare rând reprezintă o entitate individuală.

De exemplu, într-un sistem de anunțuri online, un tabel numit „anunturi” poate conține coloane precum: id, titlu, descriere, preț, categorie și dată_publicare. Fiecare rând reprezintă un anunț distinct.

În aplicații simple, datele pot fi gestionate manual în fișiere statice (de exemplu pagini HTML). Totuși, pe măsură ce volumul de date crește, această abordare devine inefficientă și predispusă la erori. Utilizarea unei baze de date permite:

- actualizarea rapidă a informațiilor;
- evitarea duplicării datelor;
- accesul simultan al mai multor utilizatori;
- menținerea consistenței și integrității datelor.

Astfel, bazele de date sunt esențiale în aplicațiile moderne, cum ar fi magazinele online, platformele de anunțuri, sistemele de evidență a studenților sau aplicațiile bancare.

MySQL reprezintă o soluție eficientă și accesibilă pentru gestionarea bazelor de date relaționale. Prin performanța sa, flexibilitatea, suportul larg și costul redus, MySQL este potrivit atât pentru mediul academic, cât și pentru dezvoltarea aplicațiilor reale. Înțelegerea principiilor de funcționare ale MySQL și ale limbajului SQL constituie o bază solidă pentru orice student sau dezvoltator interesat de lucrul cu baze de date.

7.2. Exemple de interogari SQL pe baza de date create

Ștergerea (desființarea) unui tabel

```
DROP TABLE Feedback;
```

Modificări asupra tabelelor

-Adăugarea unei coloane

```
ALTER TABLE Clienti
```

```
ADD email VARCHAR(100);
```

-Modificarea tipului unei coloane

```
ALTER TABLE Angajati
```

```
MODIFY salariu DECIMAL(10,2);
```

-Redenumirea unei coloane

```
ALTER TABLE Servicii
```

```
RENAME COLUMN durata TO durata_min;
```


-Ștergerea unei coloane

```
ALTER TABLE Clienti  
DROP COLUMN email;
```

-Adăugarea unei constrângeri

```
ALTER TABLE Clienti  
ADD CONSTRAINT unique_tel UNIQUE(nr_tel);
```

Interogări

-Lista tuturor saloanelor

```
SELECT * FROM Saloane;
```

-Toți angajații din București

```
SELECT A.nume, A.prename, S.oras  
FROM Angajati A  
JOIN Saloane S ON A.id_salon = S.id_salon  
WHERE S.oras = 'Bucuresti';
```

-Angajații care au salariul peste media salariilor

```
SELECT nume, prename, salariu  
FROM Angajati  
WHERE salariu > (SELECT AVG(salariu) FROM Angajati);
```

CAPITOLUL VIII. CONCLUZII

Baza de date realizată în cadrul acestui proiect, denumită salon_winchester, a fost proiectată pentru a gestiona activitatea unui lanț de saloane de înfrumusețare, acoperind principalele entități implicate în funcționarea acestuia: saloane, angajați, clienți, servicii, programări, furnizori, materiale și achiziții. Structura bazei de date reflectă un model relațional clasic, în care datele sunt organizate în tabele interconectate prin chei primare și chei externe.

Modelarea bazei de date a urmărit respectarea principiilor normalizării până la a treia formă normală (3NF), astfel încât să se evite redundanța inutilă și anomaliile de actualizare. De exemplu, informațiile despre clienți sunt stocate separat de programări, iar legătura dintre acestea se face prin identificatori unici. În mod similar, serviciile sunt definite într-un tabel distinct și sunt asociate programărilor prin intermediul tabelului de legătură Detalii_Programari, ceea ce permite existența mai multor servicii pentru o singură programare.

Din punct de vedere funcțional, baza de date permite:

- gestionarea saloanelor și a angajaților aferenți fiecărei locații;
- evidența clienților și a programărilor acestora;
- asocierea programărilor cu unul sau mai multe servicii;
- gestionarea furnizorilor și a achizițiilor de materiale;
- urmărirea feedback-ului oferit de clienți.

Alegerea SGBD-ului MySQL a fost determinată de mai mulți factori relevanți pentru scopul proiectului. Suportul pentru limbajul SQL standard facilitează formularea interogărilor și asigură portabilitatea cunoștințelor dobândite către alte sisteme similare. Un alt avantaj important al MySQL este ușurința în instalare și utilizare, ceea ce îl face potrivit pentru implementarea locală a bazei de date pe un calculator personal, conform cerințelor proiectului.

În concluzie, baza de date salon_winchester îndeplinește cerințele funcționale ale domeniului modelat și este construită pe principii solide de proiectare relațională. Utilizarea SGBD-ului MySQL a permis implementarea eficientă a structurii propuse.

BIBLIOGRAFIE

Cursuri BD

Laboratoare BD

<https://dbdiagram.io/d/6955273739fa3db27be1c986> - schema logica

<https://app.diagrams.net/?libs=general;er#> - schema conceptuala

https://ro.wikipedia.org/wiki/Schemă_logică

<https://www.studocu.com/ro/document/academia-de-studii-economice-din-bucuresti/baze-de-date-data-bases/laborator-3-baze-de-date/14667391>